

PA-2 CS683

Team members

- Satanshu Aglawe (25m0811)
- Arpit Meher (25m0813)
- Gaurav (25m0838)

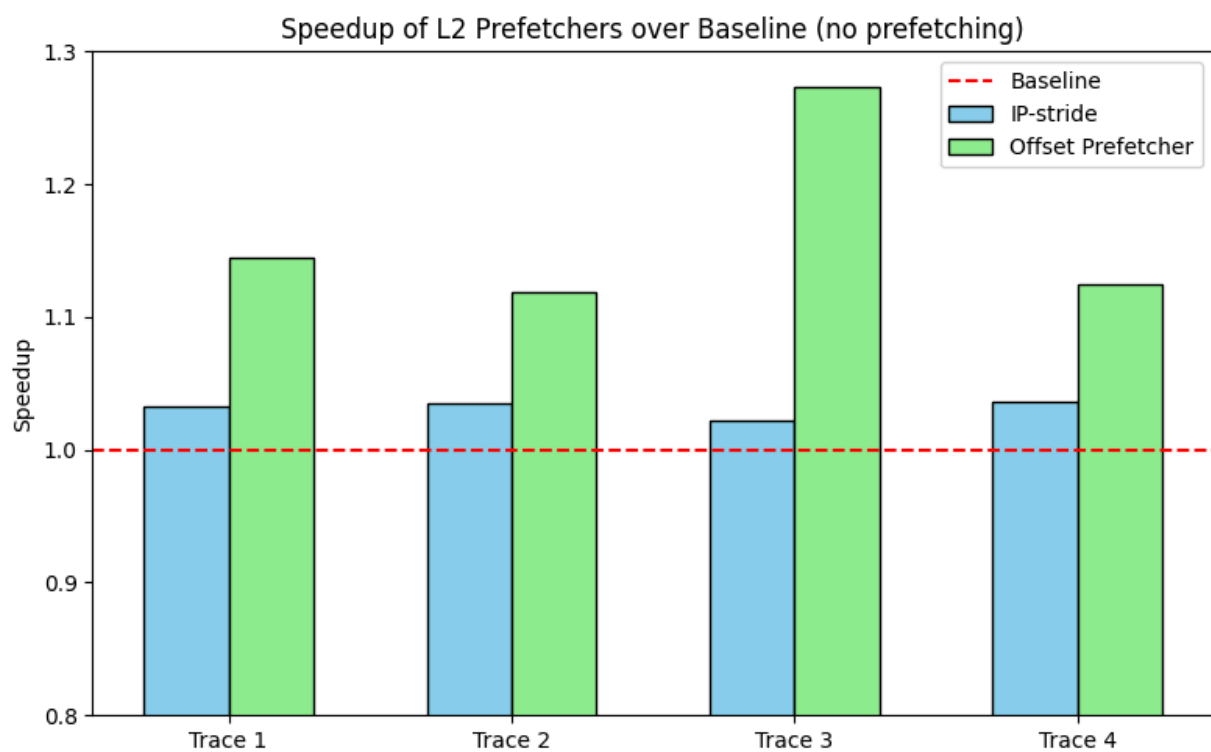
Repo link : https://github.com/satanshu5/PA2_CS683

Task 1

Table with IPC, Speedup, L2 MPKI for different table sizes.

	IPC	Speed Up	L2 MPKI
Table size = 32			
Trace 1	0.605894	1.145300987	40.3387
Trace 2	0.517251	1.118941826	47.6996
Trace 3	1.06188	1.272765846	22.1966
Trace 4	0.519857	1.124559787	47.5794
Table size = 64			
Trace 1	0.605894	1.145300987	40.3387
Trace 2	0.517251	1.118941826	47.6996
Trace 3	1.06188	1.272765846	22.1966
Trace 4	0.519857	1.124559787	47.5794

Table size = 128			
Trace 1	0.606444	1.146340634	40.3384
Trace 2	0.517059	1.118526482	47.6949
Trace 3	1.06188	1.272765846	22.1966
Trace 4	0.51996	1.124782597	47.594



Speed up plot showing comparison between IP-stride and Offset prefetcher with table size 64.

Data structures used in code

Offset

Stores an individual offset and its statistics. `offset` is used to store stride between accesses, `frequency` is used to store how often the offset is seen, `issued` is the number of prefetches issued using this offset, `useful` is the number of those prefetches that were later used.

PageEntry

represents a page region. `num` is page number (used as index), `last_line_addr` is last cache line accessed in this page, `offsets[5]` is array to store top 5 observed offsets, `lru` is least-recently-used counter for eviction.

`l2c_prefetcher_operate()`

Called on every memory access and performs the learning & prefetching logic:

1. Gets the page num from the physical address using `addr >> LOG2_PAGE_SIZE`.
2. Searches the offset table for an entry matching the current page. If not found, it allocates a new entry.
3. Then computes the stride (offset) between the current cache line and the last accessed line in the same page.
 - If the offset already exists in the page's entry, its frequency is incremented.
 - If not present, add a new entry (if no space then the least frequent offset is replaced with the new one [LRU]).
4. For each offset, calculate its accuracy as `useful / issued`.
 - Based on accuracy set the prefetch degree: 1 (low), 2 (medium), or 3 (high).
 - It then issues prefetch requests for future addresses using this offset.

`l2c_prefetcher_cache_fill()`

Called when a cache line is filled. If the fill was by a prefetch, it checks whether the filled address matches a predicted offset (and increments the useful counter for that offset).

Task 2

Procedure to check if “cache_type” if L1 (I/D):

```
C/C++
bool is_l1_cache(int cache_type) {
    return cache_type == IS_L1D || cache_type == IS_L1I;
}
```

Procedure to invalidate a cache block from L1, L2, and/or LLC cache:

```
C/C++
void invalidate_cache_entry(uint32_t cpu, uint64_t addr, bool l1, bool l2,
                           bool llc) {
    if (l1) {
        ooo_cpu[cpu].L1I.invalidate_entry(addr);
        ooo_cpu[cpu].L1D.invalidate_entry(addr);
    }

    if (l2)
        ooo_cpu[cpu].L2C.invalidate_entry(addr);
    if (llc)
        uncore.LLC.invalidate_entry(addr);
}
```

Procedure to invalidate cache block at “addr” from all cache levels except “cache_type”:

```
C/C++
void handle_invalidation(uint32_t cpu, int cache_type, uint64_t addr) {
    if (is_l1_cache(cache_type)) {
        invalidate_cache_entry(cpu, addr, false, true, true);
    } else if (cache_type == IS_L2C) {
        invalidate_cache_entry(cpu, addr, true, false, true);
    } else if (cache_type == IS_LLC) {
        invalidate_cache_entry(cpu, addr, true, true, false);
    }
}
```

Procedure to assert that a cache block (from memory access “pkt”) is absent in L1, L2, and/or LLC cache levels:

```
C/C++
void assert_block_absence_in_cache(uint32_t cpu, PACKET *pkt, bool l1, bool l2,
                                   bool llc) {
    if (l1 && (ooo_cpu[cpu].L1I.check_hit(pkt) >= 0 ||
              ooo_cpu[cpu].L1D.check_hit(pkt) >= 0)) {
        std::cerr << "[violation]: block is also present in L1";
        assert(0);
    }

    if (l2 && ooo_cpu[cpu].L2C.check_hit(pkt) >= 0) {
        std::cerr << "[violation]: block is also present in L2";
        assert(0);
    }

    if (llc && uncore.LLC.check_hit(pkt) >= 0) {
        std::cerr << "[violation]: block is also present in LLC";
        assert(0);
    }
}
```

We are invalidating the cache block from all other levels after a fill request in “handle_fill” method:

```
C/C++
fill_cache(set, way, &MSHR.entry[mshr_index]);
handle_invalidation(cpu, cache_type, MSHR.entry[mshr_index].address);
```

And finally asserting that a cache block is invalidated from all other levels in “handle_read” method:

```
C/C++
PACKET* pkt = &RQ.entry[index];

if (is_l1_cache(cache_type)) {
    assert_block_absence_in_cache(read_cpu, pkt, false, true, true);
} else if (cache_type == IS_L2C) {
    assert_block_absence_in_cache(read_cpu, pkt, true, false, true);
}
```

```

} else if (cache_type == IS_LLC) {
    assert_block_absence_in_cache(read_cpu,pkt, true, true, false);
}

```

However, this assertion logic has been commented out as it was resulting in assertion failures. There are a few more places that we need to figure out to add the same invalidation logic.

Exclusive	IPC	MPKI		
		L1D	L2C	LLC
Trace 1	0.461901	66.0504	96.2255	78.1828
Trace 2	0.398008	78.022	113.702	92.4072
Trace 3	0.764118	35.7547	52.3773	42.6186
Trace 4	0.40038	77.7235	113.289	92.0721

Speed Up	
Trace 1	0.873115
Trace 2	0.860989
Trace 3	0.915869
Trace 4	0.866105

